

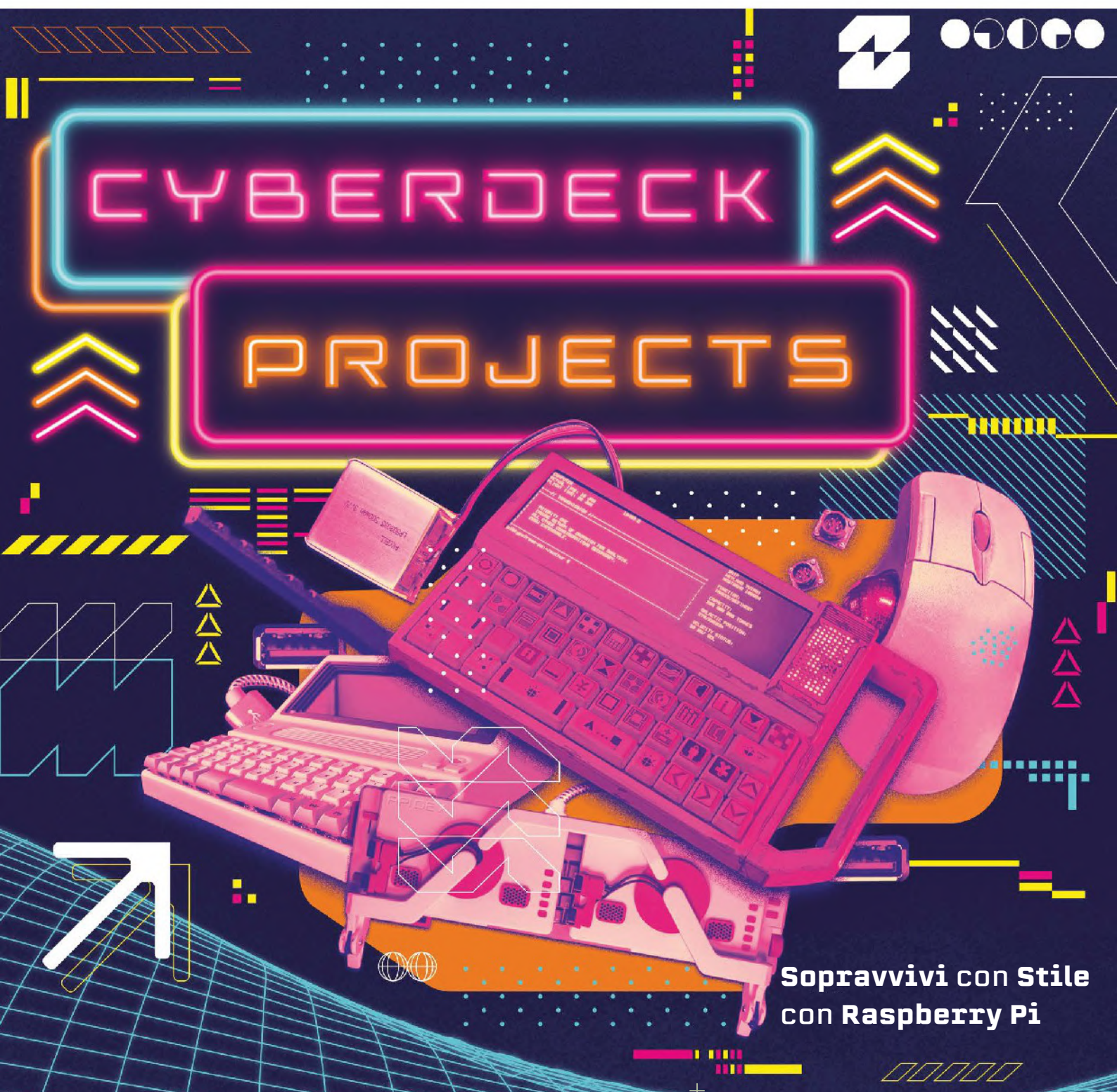
Crea
il tuo
cyberdeck

Comprendi
gli algoritmi di ricerca
con Dijkstra



Official Magazine
#168 | Agosto 2026

Raspberry Pi



**Sopravvivi con Stile
con Raspberry Pi**



Estratto dal numero 168 di Raspberry Pi Official Magazine. Traduzione di Zzed e marcolecce, revisione testi e impaginazione di Mauro "Zzed" Zoia (zzed@raspberrypi.com), per la comunità italiana Raspberry Pi www.raspberrypi.com. Distribuito con licenza CC BY-NC-SA 3.0. Raspberry Pi Official Magazine è pubblicato mensilmente da Raspberry Pi Ltd., 194 Cambridge Science Park, Milton Road, Cambridge, England, CB4 0AB.



CYBERDECK

PROJECTS

Scopri i computer di ispirazione
cyberpunk realizzati con
Raspberry Pi e vari componenti.

DI ROSIE HATTERSLEY





In un mondo costantemente connesso, perché accontentarsi di un normale telefono o laptop quando informazioni, intrattenimento, comunicazione e giochi illimitati possono essere facilmente a tua disposizione con un semplice tocco su una console fai-da-te con pulsanti e menu personalizzati, progettati secondo la fantasia che preferisci?

Dimentica le noiose lastre di vetro e colla. I cyberdeck si ispirano ai computer del mondo cyberpunk. Tastiere rumorose, righe di codice a terminale e una costruzione pensata per resistere a una distopia futuristica incredibilmente realistica.

I cyberdeck, creati come apparecchio di elaborazione mobile e accesso online, stanno vivendo un momento di grande popolarità. Già diffusi tra gli utenti di Reddit, gli appassionati di fai-da-te e gli amanti del cyberpunk, sono stati recentemente presentati sulla celebre emittente televisiva CNN dopo un'impennata di popolarità sui social media. Molti cyberdeck sono basati su Raspberry Pi; infatti, nel corso degli anni abbiamo presentato numerose creazioni sulle pagine della nostra rivista e online (rpimag.co/cyberdeckceleb).

Qui, rendiamo omaggio ad alcuni eccellenti esempi di cyberdeck che mettono in mostra l'individualità e la creatività dei loro creatori, oltre a rendere omaggio alle origini cyberpunk del concetto. Esamineremo anche alcuni dei componenti necessari per iniziare a costruire il tuo cyberdeck. Infine, Brian Corteil ci mostrerà come ha costruito il suo.

I computer presenti nel mondo del genere cyberpunk

Progetti cyberdeck

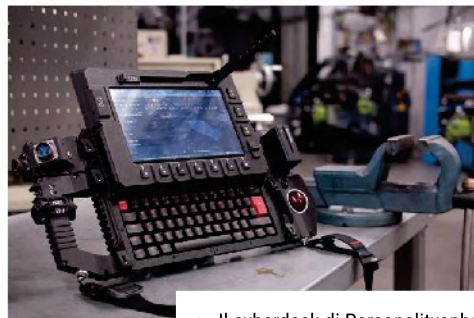
Le robuste scocche dei primi portatili e dei computer a conchiglia costituiscono degli alloggiamenti ideali per i cyberdeck, mentre altri hanno origini completamente diverse.

Una breve storia del netrunning

I cyberdeck hanno origine nella fantascienza e nella narrativa speculativa degli anni '80 e '90. Il romanzo *Neuromante* di William Gibson del 1984 ha avuto un'enorme influenza sul nostro modo di pensare ai computer e ha dato origine al concetto di cyberpunk. Nel mondo di Gibson, i cyberdeck offrono ai "cowboy delle console" o agli hacker la possibilità di connettersi direttamente a una Matrice globale tramite dermatrodi (sensori sulla fronte). Oltre all'estetica cyberpunk, questi film, romanzi e videogiochi hanno ispirato centinaia di maker nel realizzare cyberdeck fai-da-te con cui "connettersi alla rete", il più delle volte con Raspberry Pi come componente centrale.

Il cyberpunk ha anche dato origine ai Netrunner, hacker potenziati ciberneticamente che scoprono sistemi da hackerare e vendono le informazioni che scoprono ai fixer. Il nome deriva da un popolare gioco di carte omonimo del 1996 (rpimag.co/netrunnerlife). I cervelli dei netrunner si connettono direttamente alla rete utilizzando mezzi di accesso più efficienti rispetto alle tastiere tradizionali.

Tuttavia, la maggior parte dei cyberdeck presenta sia una tastiera che uno schermo, con design che richiamano i primi anni dei computer portatili e degli insoliti dispositivi palmari a conchiglia, e sono generalmente progettati per la portabilità e per resistere a potenziali attacchi da parte di bersagli virtuali o reali.



▲ Il cyberdeck di Personalitysphere viene utilizzato per la progettazione con FreeCAD

Cyberdeck Personalitysphere

rpimag.co/personalitysphere

Hacker alias: Personalitysphere

Il maker Personalitysphere ha deciso di sacrificare la portabilità per un design interessante. Questo cyberdeck basato su Raspberry Pi 5 da 8 GB è dotato di guide Picatinny per tenere in posizione la fotocamera e il trackpad, utilizzando una versione economica e non convenzionale dei connettori LEMO 0B, impiegati da fotografi e videomaker per bloccare gli oggetti ad angolazioni specifiche. Funziona con una batteria sostituibile da fotocamera. Far scorrere lo schermo verso l'alto prima di iniziare a digitare è stato un aspetto particolarmente piacevole per il maker. Sebbene non offra il miglior angolo di visuale, Personalitysphere utilizza con soddisfazione il suo computer cyberdeck per i progetti FreeCAD. È sufficientemente robusto da poter essere utilizzato nella sua officina metallurgica, calda e sporca.

Hackberry Pi

rpimag.co/hackberrypi

Hacker alias: Paul Asadoorian

▼ Il kit Hackberry Pi è basato sul Compute Module 5.

La portabilità, unita alla possibilità di utilizzarlo come strumento diagnostico e per i test di penetrazione, ha rappresentato una chiara motivazione al maker Paul Asadoorian per la realizzazione del suo "cyberdeck definitivo", basato su Hackberry Pi Elecrow di ZitaoTech (rpimag.co/hackberrypicm5), un SSD NVMe e un

Raspberry Pi Compute Module 5 con 16 GB di RAM. Ha chassis in alluminio, uno schermo da 720 × 720 pixel e una batteria da 5000 mAh per una autonomia eccezionale.

È assemblato utilizzando parti esistenti, anziché essere progettato su misura ed è dotato di due porte USB 3.0 per aggiungere connettività Bluetooth o una chiavetta Wi-Fi.





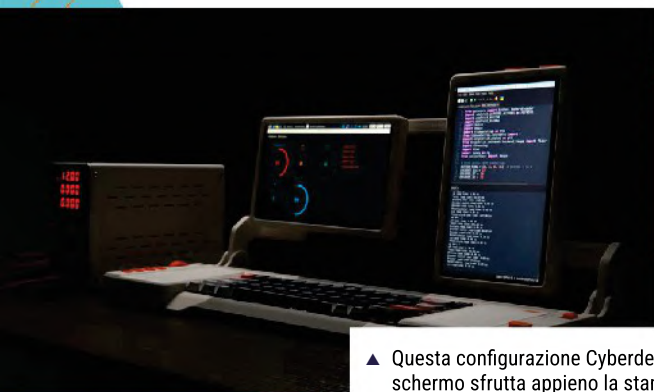
Amstrad PPC 640 cyberdeck

rpimag.co/ppc640

Hacker alias: Mikeydamager

Quando Mikeydamager ebbe bisogno di mostrare una narrazione elettronica, decise di smontare un vecchio Amstrad PPC 640 per questo scopo. Il venerabile PC compatibile IBM fu lanciato nel 1987, con un prezzo superiore a 1000 dollari, e alcuni esemplari si vendono su eBay a 250 sterline o più. Tuttavia, Mikey non aveva bisogno di un modello vintage funzionante: voleva solo un dispositivo accattivante per attirare l'attenzione e invogliare le persone a leggere la sua letteratura interattiva, un obiettivo a cui contribuiscono anche i LED "malinconici". Un Raspberry Pi 3 con una nuova scheda driver ha fornito il cervello del sistema, mentre un vecchio tablet ha svolto la funzione di schermo ed era perfetto per l'alloggiamento originale dell'Amstrad. Dopo aver collegato alcuni interruttori e manopole per le batterie, ha scollegato uno dei lettori di floppy disk e ha collegato il suo interruttore di protezione dalla copia a un Arduino per attivare lo schermo all'inserimento di un disco. Questo avvia l'app React che mostra la sua "narrazione immersiva e interattiva". Ammette che "c'è un piccolo trucco scenico in atto". Considerando che i cyberdeck sono nati come oggetti di scena fantascientifici, pensiamo che sia più che giusto.

◀ Questo straordinario progetto ricicla un vecchio PC compatibile IBM



▲ Questa configurazione Cyberdeck a doppio schermo sfrutta appieno la stampa 3D personalizzata

Dual-screen cyberdeck

rpimag.co/dualcyberdeck

Hacker alias: Sector 07

L'attenzione ai dettagli del maker Austin (alias Sector 07) nella progettazione e nella descrizione del suo incredibile cyberdeck a doppio schermo è semplicemente meravigliosa. Il cyberdeck, progettato meticolosamente e interamente stampato in 3D (a eccezione del Raspberry Pi 4 al suo interno), si assembla a mano, senza bisogno di attrezzi, ma può anche essere smontato con facilità. Il Raspberry Pi 4 trova perfettamente posto nella parte destra della tastiera personalizzata, dove sono presenti un cursore lineare per il controllo del volume, un controller rotativo programmabile e altre manopole e pulsanti accattivanti. Ancora meglio, i pin GPIO sono accessibili, consentendo di aggiungere un HAT personalizzato.

C'è un piccolo trucco scenico in corso

Smith Corona cyberdeck

rpimag.co/smithcorona

Hacker alias: Maz_Baz

Questa trasformazione basata su Raspberry Pi 4B è stata ispirata dai terminali per computer "vecchia scuola [degli anni '80]", nonché da Star Wars e dai cyberdeck. La tastiera "deteriorata" del Mark XL PWP del 1989 necessitava di essere sostituita, l'unità disco era rotta e lo schermo era difettoso, ma Maz_Baz era attratto dal potenziale "vecchia tecnologia, nuovo potenziale" della macchina da scrivere portatile Smith Corona. Un alimentatore Anker si è inserito perfettamente nello scomparto originale e il case era abbastanza spazioso per un Raspberry Pi 4B, una tastiera Vortex al 40% e un ampio schermo Waveshare da 7,9 pollici. Gli aspetti più impegnativi sono stati la "frustrante e necessariamente ingegnosa" gestione dei cavi e la misurazione e stampa 3D precise del nuovo chassis della tastiera e delle staffe di supporto per il monitor.

▼ Questa macchina da scrivere Smith Corona si presta a un progetto di cyberdeck innovativo.



Tecnologia datata, nuovo potenziale

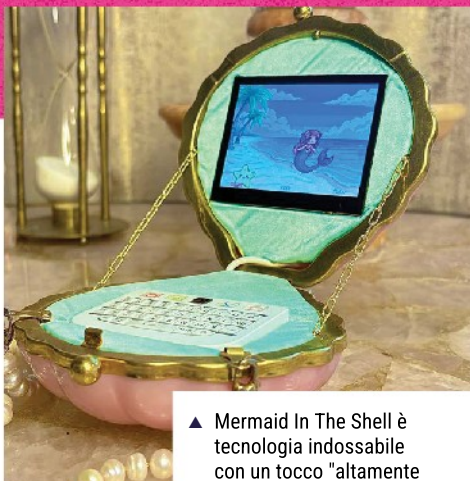
Clamshell cyberdeck

rpimag.co/clamshellblackberry

Hacker alias: Michael Klements

Si dice che il BlackBerry a conchiglia sia tornato di moda. Questa versione esegue Raspberry Pi OS su Raspberry Pi 4, è dotata di una tastiera Solder Party in stile BlackBerry ed è abbinata a uno schermo HyperPixel 4.0, alimentato da un pin GPIO. Il maker Michael ha trovato Raspberry Pi "perfetto" per il suo progetto, poiché l'ingombro della tastiera è inferiore a quello del display, mentre la scheda di interfaccia personalizzata Solder Party tratta il trackpad e la tastiera come dispositivi USB, consentendo comunque di ottenere la piena funzionalità di mouse e tastiera. A completamento della configurazione, Michael fornisce istruzioni dettagliate per la creazione del proprio case in compensato: rpimag.co/maketinycyberdeck.

- ▼ L'insolita custodia in compensato riduce al minimo il peso della confezione a conchiglia in stile BlackBerry.



- ▲ Mermaid In The Shell è tecnologia indossabile con un tocco "altamente femminile".

Mermaid In The Shell cyberdeck

rpimag.co/mermaidcyberdeck

Hacker alias: Claudia Bimbotech

Questo grazioso design a conchiglia offre un'interpretazione davvero diversa del cyberdeck, utilizzando la

struttura naturale di una pochette a conchiglia per ospitare il 'deck' fantasy della maker. Claudia desiderava che il suo cyberdeck fosse l'opposto di una custodia Pelican "da uomo", scegliendo una pochette rosa e una finitura "altamente femminile". Il suo cyberdeck Mermaid è dotato di Raspberry Pi Model A+, un touchscreen da 3,5 pollici, una tastiera e un touchpad Bluetooth ZitaoTech Q10 in stile BlackBerry con Bluetooth, collegati tramite magneti al neodimio e alimentati da una batteria da 3000 mAh.

Claudia ci teneva che il suo cyberdeck fosse l'opposto di una valigia Pelican "da uomo".

Backpack Cyberdeck

rpimag.co/backpackcyberdeck

Hacker alias: Davide Marchetti

Questo pratico zaino ospita un intero laboratorio portatile che il suo maker, appassionato ciclista, può facilmente portare da un luogo all'altro, interagendo con ogni tipo di dispositivo durante i suoi spostamenti. Davide lo usa come laboratorio mobile per la risoluzione dei problemi, con una radio GNU per eseguire la diagnostica delle comunicazioni wireless e strumenti Kali Linux per la sicurezza e i test di penetrazione. Il Backpack Cyberdeck utilizza un Raspberry Pi 4 "leggermente overclockato" alloggiato in una struttura protettiva ma leggera in compensato e stampa 3D che mantiene tutto saldamente in posizione. La configurazione può essere utilizzata da remoto e rispetta le linee guida di sicurezza aerea per la potenza erogata, quindi può essere portata a bordo anche degli aerei.

- ◀ Questa borsa è un laboratorio diagnostico portatile in uno zaino



Alien Typeframe PS-85 cyberdeck

rpimag.co/typeframes85

Hacker alias: Jeff Merrick

Partendo da un set di tasti Semiotic Symbols basati sui simboli futuristici di Ron Cobb in Alien, il cosplayer Jeff ha iniziato a creare un cyberdeck ispirato al computer Epson PX-4 del 1985, simile a una macchina da scrivere. Per il progetto basato su Raspberry Pi 4, Jeff ha optato per la semplicità e ha scelto un formato a tavoletta perché le cerniere complicano tutto, oltre a un display Waveshare da 7,9 pollici. Lasciando un connettore GPIO esposto, ha aggiunto un Adafruit LED Matrix Bonnet, che rappresenta il sistema madre AI MU/TH/UR di Alien. Forte della sua esperienza nella creazione di oggetti di scena per cosplay, Jeff ha aggiunto una maniglia distintiva e ha personalizzato l'esterno del cyberdeck con una verniciatura che includeva il dentifricio per dare un aspetto più vissuto alle aree che voleva far apparire usurate.

- ▼ Simboli semiotici e un autentico effetto invecchiamento caratterizzano questo eccellente cyberdeck ispirato ad Alien.



La tipica postazione di combattimento degli antieroi "cowboy della console" che percorrono le reti attraverso il cyberspazio

PiDex Doomsday Companion

rpimag.co/pidex

Hacker alias: Raspduino Uno

Kenneth, soprannominato "Raspduino Uno", è una sorta di prepper, con una scatola robusta e impermeabile in cui ha installato un Raspberry Pi 5 e ogni sorta di tecnologia per sopravvivere a qualsiasi evenienza. Un touchscreen da 7 pollici, una mini tastiera retroilluminata (per leggere al buio), un hub USB a quattro porte, un power bank, un pannello solare, una copia di Wikipedia su una chiavetta USB separata e diversi pulsanti rossi di grandi dimensioni dall'aspetto importante lo rendono un kit degno di qualsiasi survivalista.



- ▼ I prepper si prepareranno con questo cyberdeck a energia solare, progettato per l'uso autonomo.



Crea il tuo deck

Componenti e kit per costruire la propria attrezzatura portatile

COMPUTER: RASPBERRY PI

Avrai bisogno di un computer per assemblare il tuo cyberdeck. Un Raspberry Pi 5 è l'ideale, ma il Raspberry Pi 4 è un'alternativa più economica che ti offrirà comunque tutta la potenza necessaria.

rpimag.co/raspberrypi5



SMARTIPI TOUCH PRO 3:

Se ti piace l'idea di utilizzare il tuo cyberdeck come hub di controllo centrale, questo robusto case con touchscreen da 7 pollici è un'ottima scelta: offre spazio sufficiente non solo per il display del Raspberry Pi, ma anche per un Camera Module e per eventuali HAT collegati al Raspberry Pi stesso. Aggiungi qualche luce per un effetto perfettamente fantascientifico.

rpimag.co/smartipipro3



TASTIERA: KEEBDECK BASIC

Scegliere una tastiera è una questione molto personale, ma l'aspetto di questa KeebDeck ci piace. Si tratta di una tastiera portatile già assemblata, facile da integrare nelle proprie configurazioni cyberdeck.

rpimag.co/keebdeck



SCHERMO: HYPERPIXEL 4.0

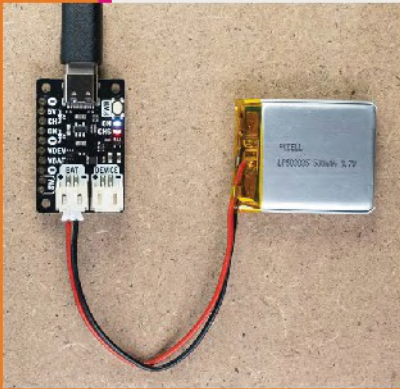
Dai al tuo cyberdeck uno schermo. Le opzioni tra cui scegliere sono molte, ma ecco l'HyperPixel 4.0 di Pimoroni. Si tratta di un display TFT ad alta risoluzione da 4 pollici per Raspberry Pi, disponibile anche con funzionalità multi-touch capacitivo opzionale. Si collega tramite il connettore GPIO ed è la soluzione ideale per aggiungere uno schermo a un Raspberry Pi.

rpimag.co/hyperpixel4



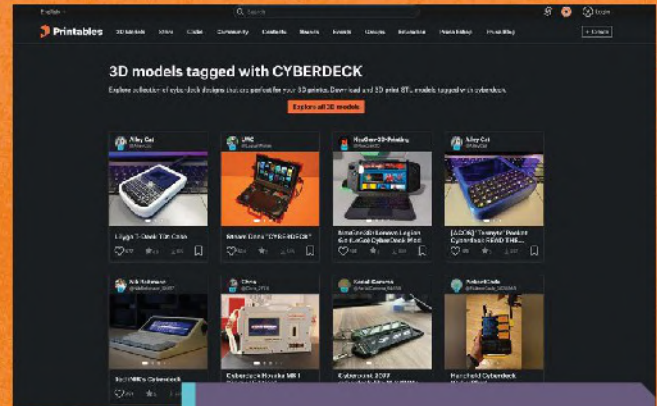
ALIMENTAZIONE: CARICABATTERIA LIPO AMIGO

Avrai bisogno di alimentare il tuo cyberdeck anche in movimento. Quindi scegli un caricabatterie per batterie LiPo (polimeri di litio) di alta qualità, come il LiPo Amigo. Questa scheda, grande quanto un francobollo, ha un connettore USB-C a una estremità e un connettore JST-PH all'altra, per poter collegare una batteria LiPo o agli ioni di litio.
rpimag.co/lipoamigo



SB COMPONENTS LORA HAT

Questo accessorio da 27 sterline offre connettività tramite una rete LoRaWAN, consentendoti di collegare il tuo cyberdeck per inviare e ricevere messaggi e dati in luoghi dove il Wi-Fi non arriva. Operando su tre frequenze wireless globali, il LoRa HAT è dotato di una antenna a stelo e funziona con qualsiasi Raspberry Pi con GPIO a 40 pin di dimensioni standard. Le informazioni raccolte, come i dati ambientali, possono essere inviate in modalità wireless e sicura, permettendoti di avvisare i tuoi contatti sulle condizioni di vita nei luoghi in cui ti trovi con il tuo cyberdeck.
rpimag.co/lorahat



CASE: STAMPA 3D DA PRINTABLES

Praticamente qualsiasi cosa può essere riciclata e trasformata in una custodia. Ma se ne desideri una stampata in 3D personalizzata, dai una occhiata alla gamma disponibile su Printables.

rpimag.co/cyberdeckcases

ENVIRONMENT SENSOR HAT DI WAVESHARE:

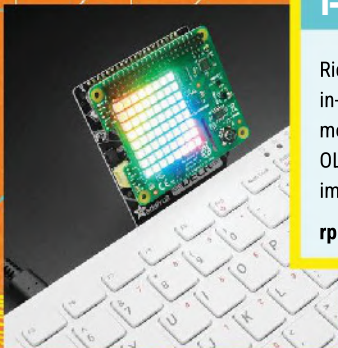
Da non confondere con il Sense HAT, che è stato nello spazio a bordo della ISS e offre una straordinaria gamma di effetti visivi basati su ciò che rileva, l'Environment Sensor HAT fornisce preziose informazioni su pressione atmosferica, umidità, livelli di luce ambientale e UV, oltre ad avvisarti di pericoli derivanti da solventi, vernici e particolato. Un giroscopio e un accelerometro, insieme agli avvisi di temperatura, lo rendono un compagno affidabile quando tu e il tuo cyberdeck vi avventurate in territori sconosciuti.
rpimag.co/envsenhat



All-in-one: Cyberdeck HAT per Raspberry Pi 500/400

Riduci la costruzione e collega uno schermo al computer all-in-one Raspberry Pi con l'HAT Cyberdeck di Adafruit per i modelli Raspberry Pi 500 e 400. Puoi usarlo insieme a un OLED Bonnet (o un HAT simile) per ottenere un display immediato.

rpimag.co/cyberdeckhat



Crea il tuo cyberdeck

Un vero e proprio rito di passaggio per un hacker è costruire il proprio cyberdeck. Questa guida ti aiuterà a realizzarne uno assemblandone i componenti.



Maker

Brian Corteil

Creatore di due zootropi digitali, un cappello unicorno gigante (realizzato con 64 cappelli unicorno), un numero incalcolabile di robot e un paio di cyberdeck.

corteil.co.uk

Cos'è un cyberdeck? È un computer indossabile, un computer tascabile, un centro di controllo/monitor per razzi o palloni aerostatici ad alta quota, o un archivio di conoscenze per l'apocalisse zombie? La risposta è: tutte queste cose insieme e qualsiasi altra cosa ti venga in mente. È qualcosa che crei per risolvere un'esigenza che un tablet o un computer convenzionale non possono soddisfare, o semplicemente per il gusto di creare qualcosa di personale.

Considerazioni su pianificazione e progettazione

Un cyberdeck è composto da quattro parti principali: cervello, periferiche, alimentatore e case. Analizzeremo ciascuna di esse singolarmente.

- **Cervello** - Il cervello di un cyberdeck comprende l'unità di elaborazione e la memoria a lungo termine. Per quanto riguarda l'unità di elaborazione centrale (CPU), è importante considerare il consumo energetico, la potenza di elaborazione e le dimensioni. Un computer a scheda singola (SBC) è ideale per questo scopo, in quanto consuma meno energia rispetto a una CPU per PC convenzionale. La gamma Raspberry Pi offre alcune valide opzioni per un cyberdeck ed è una scelta popolare per le realizzazioni. Per l'archiviazione, Raspberry Pi offre diverse opzioni: scheda SD, unità flash USB, disco rigido USB, SSD USB o unità M.2 NVMe con un HAT M.2 se si utilizza un Raspberry Pi 5.





▲ Questa discreta valigetta in stile Pelican può essere utilizzata per il cyberdeck dei tuoi sogni

▼ Il cyberdeck completo è pronto per la tua vita fuori dalla rete. Hackera il pianeta!

▼ Il sistema di alimentazione, la batteria RTC e una piastra per montare un Raspberry Pi vengono aggiunti prima alla base.

I COMPONENTI:



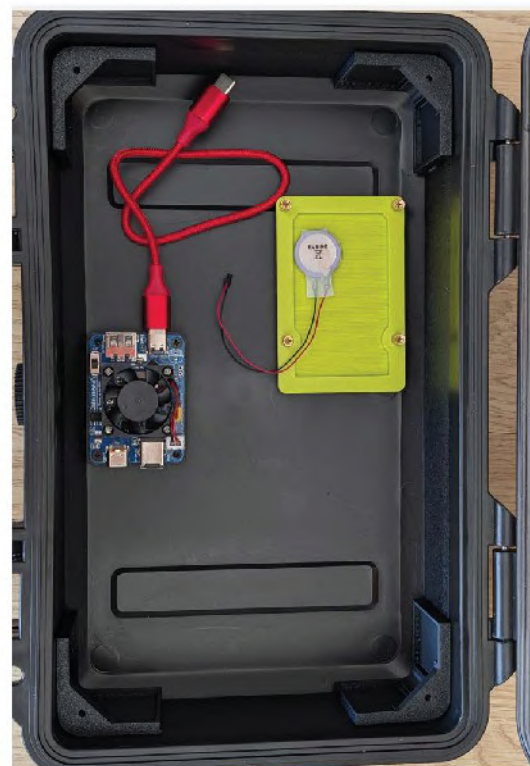
rpimag.co/cyberdeck

La gamma Raspberry Pi offre buone opzioni per un cyberdeck.

- **Periferiche** - Le periferiche sono gli strumenti con cui si interagisce con il cyberdeck. Possono essere dispositivi standard come tastiera, mouse, display, scheda audio, webcam, ecc. Oppure dispositivi controllati tramite comandi vocali, un dispositivo di puntamento integrato nel case, un joystick per il mouse, un encoder per la rotellina di scorrimento e trigger per i pulsanti del mouse. Un piccolo hub è utile se si collegano molti dispositivi USB. A seconda di come si espongono le porte della SBC, potrebbero essere necessarie estensioni per le porte da pannello.
- **Alimentazione:** l'alimentatore deve essere sufficientemente potente da alimentare l'SBC (computer su singola scheda), le periferiche e gli accessori. Per calcolare la potenza necessaria, somma i requisiti di alimentazione di ciascun dispositivo. Queste informazioni sono reperibili nelle schede tecniche o nel manuale del dispositivo.

NOTA!

Sebbene Raspberry Pi 5 richieda 25 W, non assorbe tutta questa potenza a meno che ogni porta USB non assorba la potenza massima. Se si prevede di alimentarlo dalla rete elettrica, l'alimentatore ufficiale Raspberry Pi da 27 W con porta USB-C sarà più che adatto a questa configurazione. (rpimag.co/powersupply).

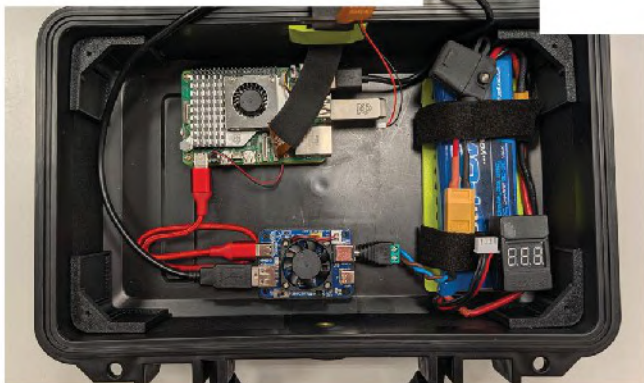


Alimentazione

- Batteria: rpimag.co/cdbattery
- prolunga XT60: rpimag.co/xt60
- Battery monitor: rpimag.co/battmon
- XT60 adapter: rpimag.co/xt60adapt
- Power module: rpimag.co/powermod

Usa un fusibile da 5A o meno nella prolunga XT60 e carica la batteria in un sacchetto ignifugo.

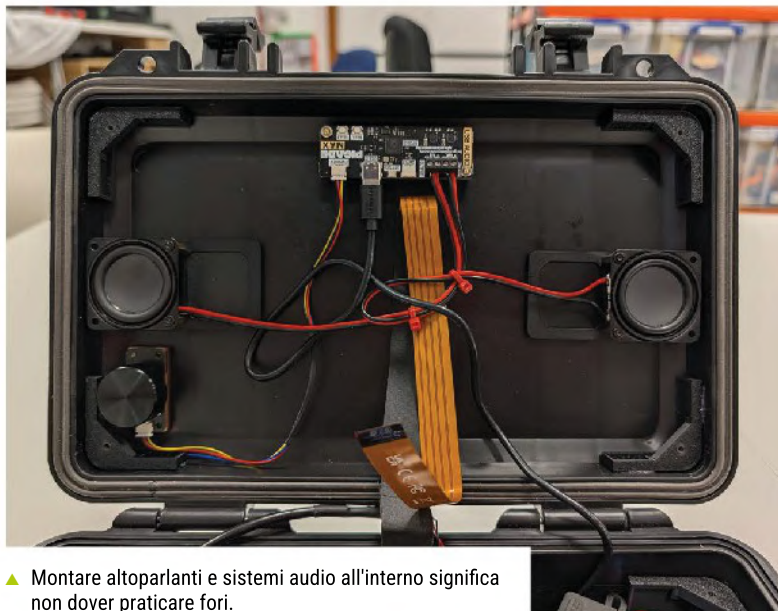
- ▼ Stratifica il cyberdeck con il Raspberry Pi... e anche la batteria



Ad esempio, un Raspberry Pi 5 richiede 25 W; ovvero 5 V a 5 A se vuoi utilizzare le porte USB alla massima potenza (questo è lo scenario peggiore, con le porte USB al massimo). Il display alla massima luminosità assorbe circa 3 W. Pertanto, è necessario un alimentatore di almeno 28 W. Ciò si traduce in circa 5,6 A.

Questo valore è stato calcolato con la formula $I = P/V$, dove I = corrente (A), P = potenza (W) e V = tensione (V), quindi $5,6 A = 28 W / 5 V$. Le opzioni per l'alimentazione includono un HAT di alimentazione o UPS, una scheda convertitore DC-DC alimentata a batterie, un alimentatore ausiliario o un power bank. Un cyberdeck non necessita di essere alimentato a batterie.

Alcuni tipi di batterie, in particolare le LiPo e simili, possono creare problemi: trattale con cura, assicurati che vengano caricate correttamente e non scaricarle eccessivamente. È consigliabile installare un fusibile il più vicino possibile alla batteria e monitorare la tensione delle celle della batteria durante l'utilizzo.



- ▲ Montare altoparlanti e sistemi audio all'interno significa non dover praticare fori.

- **Custodia** – La custodia definisce il cyberdeck. Le realizzazioni precedenti hanno utilizzato vecchi computer tascabili (inclusa la tastiera originale), una valigetta, una scatola di latta per mentine, scatole di legno e repliche di oggetti di scena. Una buona custodia di partenza è un guscio rigido in stile Pelican. Sono ampiamente disponibili in una varietà di dimensioni e fasce di prezzo. Costituiscono una custodia robusta, di solito hanno una maniglia per il trasporto e sono resistenti alle intemperie.
- **Costruzione** – Un case può essere costruito utilizzando metodi di prototipazione rapida, stampa 3D, taglio laser, pannelli in schiuma o fogli di plastica, e persino cartone. Il case deve contenere tutti i componenti e non dimenticare il cablaggio. La costruzione del tuo cyberdeck dipende dagli strumenti e dai servizi a cui hai accesso. La stampa 3D e il taglio laser sono particolarmente adatti per la costruzione del tuo cyberdeck, consentendoti di personalizzare il design del case. Il profilo in alluminio estruso da 10 mm è un ottimo supporto per lo chassis e, se utilizzato con pannelli stampati in 3D o tagliati al laser, crea un case robusto e versatile. Penne per stampanti 3D, plastica modellabile, fogli di plastica e compensato possono essere utilizzati per realizzare pannelli, supporti e qualsiasi altro elemento. Le aziende offrono servizi di stampa 3D e taglio laser se non hai accesso all'attrezzatura.
- **Componenti aggiuntivi** – Una parte importante di un cyberdeck sono i componenti aggiuntivi. Completano e definiscono la configurazione. Interruttori, LED, radio a definizione software (SDR), pin di I/O accessibili per la modifica, una porta I2C per aggiungere sensori e secondi display; un Raspberry Pi Pico per fungere da secondo processore per monitorare interruttori, sensori, pilotare un display secondario, controllare i LED e qualsiasi altra cosa si possa immaginare.



Raccolta dei componenti

Per questa guida, abbiamo selezionato i seguenti componenti, tutti reperibili in commercio. Sarà necessaria qualche saldatura oppure, in alternativa, è possibile utilizzare i blocchi Wago.

- **Raspberry Pi 5:** questa versione di Raspberry Pi è perfetta per un cyberdeck in quanto è abbastanza potente per il controllo di altri progetti Raspberry Pi, la programmazione e le attività quotidiane. Dispone inoltre di buone opzioni di connettività e dei pin GPIO per i progetti. rpimag.co/raspberrypi5
- **Raspberry Pi 128GB flash drive:** Queste unità flash sono più veloci delle schede microSD, più robuste, hanno dimensioni fisiche maggiori e quindi è meno probabile che vengano perse, e sono facili da sostituire, soprattutto se vuoi usare un sistema operativo diverso, come un media center o un OS per videogiochi retrò rpimag.co/flash.
- **Raspberry Pi 7" Touch Display 2:** Ha una risoluzione di 1280 × 720 (orientamento orizzontale). Il touchscreen elimina la necessità di un mouse nella maggior parte dei casi. rpimag.co/touchdisplay2
- **Rii RK302 Bluetooth Keyboard:** È stata scelta perché si adatta alla custodia utilizzata in questo tutorial e ha un profilo basso. È progettata per essere posizionata su un tavolo durante l'uso e il suo chassis è flessibile e necessita di supporto quando è all'interno della custodia. Ha il vantaggio di potersi connettere tramite Bluetooth. rpimag.co/rk302
- **Pimoroni Picade Max USB Audio:** Si tratta di una scheda audio USB-C compatta con un amplificatore stereo da 3W, una soluzione a un cavo/PCB, con il vantaggio di mantenere libero il connettore GPIO a 40 pin. È stato incluso anche il codificatore Picade Max (rpimag.co/picadeaudioenc) per il controllo del volume. rpimag.co/picadeaudio
- **Alimentazione:** L'utilizzo di un power bank semplifica la configurazione ed è un'opzione più sicura per alimentare un cyberdeck rispetto all'utilizzo di una batteria LiPo. Tuttavia, in questa configurazione verrà utilizzata una batteria LiPo, con un fusibile in linea e un convertitore DC-DC che eroga 5V 5A tramite USB-C PD. Come accennato in precedenza, è possibile utilizzare un alimentatore USB-C da 27W per Raspberry Pi (rpimag.co/powersupply) per alimentare il cyberdeck dalla rete elettrica. Per semplificare le cose, in questa configurazione verrà utilizzato un caricabatterie esterno per la ricarica della batteria, che verrà scollegata durante la ricarica.
- **Guscio stile Pelican:** Questa è una custodia impermeabile CASOMAN da 11,5 pollici. Il marchio Pelican è noto per le sue custodie di alta qualità ed è diventato sinonimo di robuste custodie impermeabili, proprio come Hoover lo è per gli aspirapolvere. rpimag.co/casoman
- **Raspberry Pi RTC Battery:** Aggiungendo la batteria al Raspberry Pi 5, si attiverà l'RTC integrato che manterrà l'ora anche quando il computer è offline. È necessario aggiungere questa riga al file `/boot/firmware/config.txt` affinché la batteria si carichi:

- ▼ Una piastra copre i componenti elettronici interni, in modo da poter facilmente montare una tastiera sopra.



*L'uso di un power bank
semplifica l'organizzazione
ed è un'opzione più sicura.*

```
dtparam=rtc_bbat_vchg=3000000
```

La batteria si trova qui: rpimag.co/rtcbattery.

- **Cavi USB:** Per il cablaggio interno, consigliamo i cavi piatti a nastro personalizzati disponibili da Adafruit (adafruit.com) e The Pi Hut (thepihut.com). È possibile combinarli tra loro: ad esempio, un micro HDMI ad angolo retto con un connettore HDMI standard. Per questa realizzazione, utilizziamo cavi USB standard e un cavo a nastro da 500 mm per collegare lo schermo.

CONSIGLIO RAPIDO

Il nastro in tessuto utilizzato per il cablaggio elettrico delle auto è ottimo per rivestire il cavo del display e abbinarlo alla custodia.





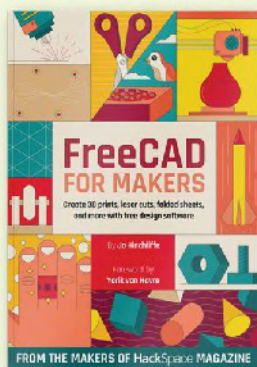
▲ La tastiera può essere qualsiasi cosa tu voglia, purché stia nello spazio

Costruire il tuo cyberdeck

0. Raccogli i componenti necessari per l'assemblaggio, installa l'OS scelto sulla memoria (puoi usare Raspberry Pi Imager per farlo: rpimag.co/imager), quindi collega i componenti e verifica che tutto funzioni correttamente.
1. Il CAD (Cardboard-Aided Design in questo caso) aiuta a progettare il tuo cyberdeck, a prototipare il layout, a capire come i componenti si assemblano e quanto spazio è necessario per il cablaggio.
2. Poi convertiamo le nozioni apprese nella fase 1 in modelli CAD (tradizionali). Abbiamo utilizzato la versione Maker di Solidworks (rpimag.co/solidworksmaker), ma ci sono anche Fusion360 e il software open source FreeCAD.
3. Stampa, taglia al laser, costruisci i supporti come progettato, pronti per l'installazione. Potresti dover apportare modifiche ai tuoi progetti durante la costruzione.
4. Inizia dal coperchio e verifica che i componenti si incastrino correttamente. Inserisci gli altoparlanti e l'amplificatore audio USB Picade Max; l'amplificatore si installa dietro il display nella parte superiore e gli altoparlanti ai lati del display, dietro le griglie del pannello frontale. Il controllo del volume si trova dietro il pannello superiore, al centro rispetto all'altoparlante sinistro. La manopola del volume sporge dal pannello del display. Successivamente, fissa il display al pannello superiore utilizzando del nastro biadesivo o della gommapiuma per tenerlo in posizione e monta le staffe posteriori. Incolla i supporti con una pistola per colla a caldo o con del nastro biadesivo in schiuma dopo aver verificato nuovamente l'incastro. Collega i cavi degli altoparlanti ai connettori, quindi collega il cavo a nastro da 500 mm per il display e i cavi di alimentazione necessari per prolungare il cavo di alimentazione del display. Il cavo a nastro del display è rivestito con un nastro in tessuto utilizzato per i cablaggi elettrici; questo nastro sostiene il cavo e lo integra nel resto del case, abbinandosi ai cavi USB. Un cavo con connettore USB-A viene saldato al cavo di alimentazione del display; utilizza una guaina termorestringente per coprire le giunzioni. Usare un cavo USB libera i pin di alimentazione sul connettore GPIO a 40 pin per HAT e progetti. Le viti autofilettanti sono ideali per fissare il pannello del display ai supporti; altre opzioni sono l'incollaggio di dadi con colla cianoacrilica o l'utilizzo di perni termofissabili.

FreeCAD for Makers

Vuoi saperne di più su FreeCAD? Abbiamo un libro che parla proprio di questo argomento, lo trovi qui: rpimag.co/freecadbook



CONSIGLIO RAPIDO

Quando utilizzi viti autofilettanti nella plastica, fai attenzione a non spanare o danneggiare la filettatura. Durante il rimontaggio, avvita in senso antiorario fino a sentire un salto: questo indica l'inizio della filettatura. Stringi la vite normalmente.

5. Verifica che i supporti e la piastra si adattino alla metà inferiore del case. Incolla i supporti in posizione. Collega il Raspberry Pi al display e assicurati che si avvii e che il display funzioni. Potresti aggiungere un interruttore per spegnere l'alimentazione del display (deve essere in serie con gli ingressi di alimentazione del display e con l'alimentatore che alimenta il display). Quando sei

soddisfatto della posizione del Raspberry Pi e ti sei assicurato che sia il display che gli altoparlanti funzionino, fissalo in posizione. Puoi incollarlo o utilizzare del nastro in velcro. Abbiamo progettato una piastra di montaggio per il Raspberry Pi e il portabatterie. Monta i supporti per la tastiera sulla piastra della tastiera. Non fissare la piastra perché dovrai poter accedere al Raspberry Pi per accenderlo/spegnere e cambiare l'alimentatore.

6. Da qui puoi personalizzare il tuo progetto. Ci sono diversi modi per farlo. Potresti usare un Raspberry Pi Pico come secondo processore per controllare un secondo piccolo display con informazioni utili, oppure per pilotare dei LED, perché i LED, in un progetto, non sono mai troppi. Potresti aggiungere dei sensori per monitorare il consumo energetico e tenere d'occhio il livello della batteria e qualsiasi altra cosa ti venga in mente.



▲ Aggiungi lo schermo e la piastra superiore per completare l'assemblaggio

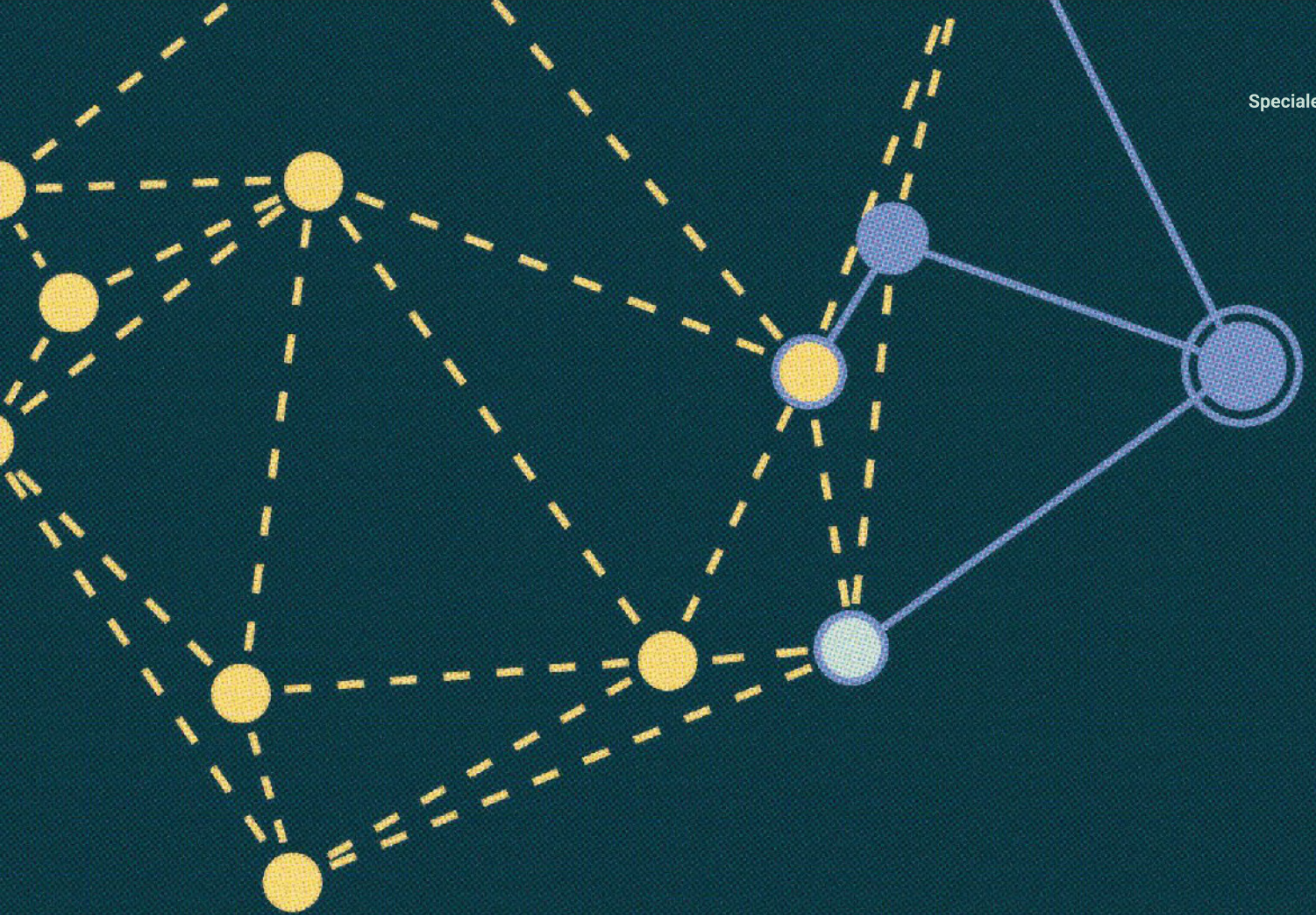


Programmazione per studenti: L'algoritmo di Dijkstra

Trova la strada verso il tesoro
utilizzando la corretta informatica.

Di Rob Miles





Uso dell'algoritmo di Dijkstra per creare dati di instradamento

Negli anni '50, l'informatico olandese **Edsger Dijkstra sviluppò un algoritmo per il routing così importante che,**

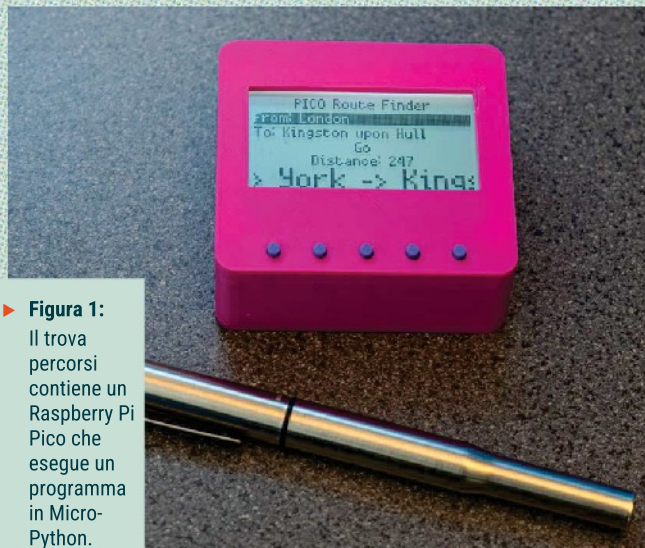
a distanza di 70 anni, fa ancora parte dei programmi di studio di informatica. In questo articolo esamineremo cosa fa l'algoritmo di Dijkstra, come funziona e dove può essere impiegato. Nel frattempo, ripasseremo alcune tecniche di programmazione in Python, troveremo un tesoro e costruiremo un dispositivo tascabile per la ricerca di percorsi.

Come fa il navigatore a trovare la strada di casa? Come decidono le reti quale percorso far seguire ai pacchetti di dati? Come può un robot orientarsi per aggirare un ostacolo? La risposta a tutte queste domande sta nella creazione di un grafo di nodi dati interconnessi e nell'utilizzo di un metodo come l'algoritmo di Dijkstra per ricavare informazioni di instradamento dal grafo stesso.

SCARICA IL
CODICE COMPLETO:



rpimag.co/dijkstracode



► **Figura 1:**
Il trova percorsi contiene un Raspberry Pi Pico che esegue un programma in Micro-Python.

L'algoritmo di Dijkstra

La **Figura 1** mostra il Pico Route Finder. È possibile trovare tutte le istruzioni per realizzarlo all'indirizzo rpimag.co/piconav. Il dispositivo contiene dati cartografici relativi a località del Regno Unito ed è in grado di indicare le distanze e i percorsi che le collegano. Vediamo come funziona grazie all'algoritmo di Dijkstra, partendo dalla storia della sua ideazione.

Si racconta che Dijkstra stesse bevendo un caffè ad Amsterdam mentre rifletteva su questioni di navigazione. Contemporaneamente, stava pensando a quale programma avrebbe potuto scrivere per dimostrare le potenzialità di un nuovo computer a cui stava lavorando. Dopo qualche minuto di riflessione, ideò il suo algoritmo, che si riassume in tre regole:

1. Tieni traccia del costo necessario per andarci.
2. Quando esplori, utilizza sempre per primo il percorso meno costoso.
3. Tieni traccia dei luoghi in cui sei stato.

Vediamo come possiamo usare queste regole per trovare il percorso più economico verso un tesoro. Immaginiamo di visitare un castello e che il proprietario ci dica:

«Voglio che tracciate un percorso dall'Atrio alla mia Sala del Tesoro, dove ho sistemato un mucchio di monete d'oro. Potrete tenere tutte le monete della Sala del Tesoro, meno il costo per arrivarci. Ogni porta lungo il percorso ha un costo. Potete girare per il castello quanto volete, ma quando avrete finito potrete raggiungermi nella Sala del Tesoro, indicarmi il percorso che avete ideato e io vi darò la vostra ricompensa».

I risultati

Ci guardiamo intorno nell'atrio e vediamo tre porte con delle etichette, come mostrato nella **Figura 2**. Le etichette riportano: "Cucina: costo 5", "Biblioteca: costo 2", "Sala del tesoro: costo 100". Potremmo andare direttamente alla Sala del tesoro, il che ci costerebbe 100 monete. Oppure potremmo tentare la fortuna cercando un percorso più economico. Fortunatamente, siamo informatici esperti dell'algoritmo di Dijkstra, quindi apriamo il nostro taccuino e disegniamo una tabella. Ogni porta ci indica una nuova stanza e il costo per raggiungerla partendo da quella posizione. Inseriamo queste informazioni nella tabella, iniziando dall'atrio, ovvero dove ci troviamo attualmente.

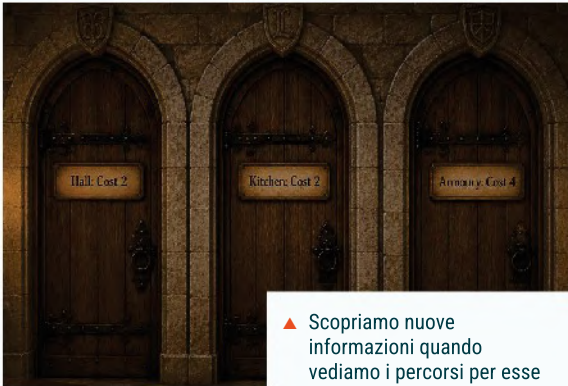
La **Figura 3** mostra la nostra prima tabella: descrive le stanze che conosciamo, indica se le abbiamo visitate, riporta il costo per ciascuna stanza dall'Atrio e specifica il percorso per tornare al punto di partenza del nostro viaggio. L'algoritmo di Dijkstra stabilisce che ora dobbiamo individuare la stanza non ancora visitata raggiungibile al costo minore, spostarci verso di essa e aggiornare la tabella. Pertanto, ci dirigiamo verso la Biblioteca.

▼ **Figura 2:** Ci si aspetterebbe che il percorso verso la Sala del Tesoro sia costoso.



Room	Visited	Cost	Route Back
Hall	yes	0	
Library	no	2	Hall
Kitchen	no	5	Hall
Treasure Room	no	100	Hall

▲ **Figura 3:** Aggiungeremo i valori nella tabella man mano che acquisiremo maggiori informazioni sui percorsi tra le stanze.



▲ Scopriamo nuove informazioni quando vediamo i percorsi per esse

Cerco oro, disposto a viaggiare

Le porte nella **Figura 4** ci indicano che raggiungere la Cucina dalla Biblioteca ha un costo di 2, mentre raggiungere l'Armeria ne costa 4. Ci segnalano inoltre che raggiungere l'Atrio costa 2, un dato che però conoscevo già grazie alla porta presente nell'Atrio stesso. Con questi valori, aggiorniamo la tabella. Partiamo dall'Armeria: raggiungerla dalla Biblioteca costa 4, e arrivare alla Biblioteca dall'Atrio costa 2; di conseguenza, il costo totale per raggiungere l'Armeria partendo dall'Atrio è 6. Inseriamo l'Armeria nella tabella e annotiamo che il percorso di ritorno verso l'Atrio, partendo dall'Armeria, prevede di passare prima per la Biblioteca e poi di proseguire verso l'Atrio.

Ora possiamo aggiornare la tabella per risparmiare. L'algoritmo di Dijkstra prevede di tenere traccia dei costi di percorrenza: se individuiamo un nuovo costo inferiore a quello già presente in tabella, dobbiamo aggiornare il valore. Raggiungere la Biblioteca costa 2, e spostarsi da lì alla Cucina ne costa altri 2, portando il totale a 4. Inizialmente, il costo per andare direttamente dall'Atrio alla Cucina era stato registrato come 5 (il valore indicato sulla porta dell'Atrio); tuttavia, passando per la Biblioteca, possiamo risparmiare una moneta d'oro. Aggiorniamo quindi la tabella con questa nuova informazione.

La **Figura 5** mostra la tabella aggiornata, con le informazioni superate sbarrate. La nostra prossima tappa sarà la stanza non ancora visitata con il costo più basso: la Cucina. Entriamo nella Cucina e ripetiamo il procedimento finché non raggiungiamo la Sala del Tesoro.

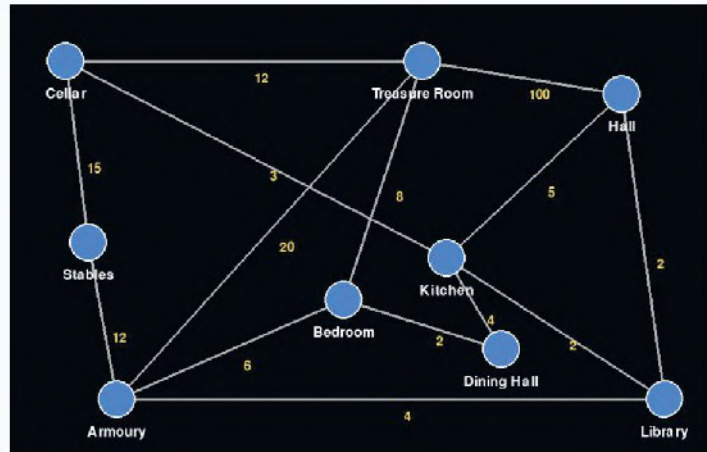
▲ **Figura 5:** Il costo per raggiungere la Cucina si è ridotto.

Room	Visited	Cost	Route Back
Hall	yes	0	
Library	yes	2	Hall
Kitchen	no	4	Hall Library
Treasure Room	no	100	Hall
Armoury	no	6	Library

Room	Visited	Cost	Route Back
Hall	yes	0	
Library	yes	2	Hall
Kitchen	yes	4	Hall Library
Treasure Room	yes	18	Hall Armoury Cellar
Armoury	yes	6	Library
Dining Hall	yes	8	Kitchen
Bedroom	yes	10	Dining Hall
Cellar	yes	7	Kitchen
Stables	no	22	Cellar

La resa dei conti finale

La **Figura 6** mostra la tabella creata durante i nostri spostamenti. Contiene tutte le info necessarie. Il percorso più economico dall'Atrio alla Sala del Tesoro ha un costo di 18 monete. Utilizziamo la colonna "Percorso di ritorno" della tabella per individuare la strada per tornare dalla Sala del Tesoro all'Atrio. La prima tappa del ritorno verso l'Atrio è la Camera da letto. Successivamente, consultiamo il valore del "Percorso di ritorno" relativo alla Camera da letto per trovare la tappa successiva, e così via. Il percorso di ritorno completo è: Camera da letto > Sala da pranzo > Cucina > Biblioteca > Atrio. Puoi seguire questo procedimento osservando la **Figura 6**. Invertendo tale percorso, otteniamo l'itinerario dall'Atrio alla Sala del Tesoro.



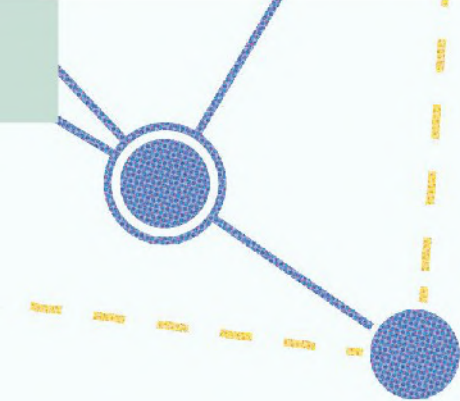
La **Figura 7** mostra la mappa del castello che abbiamo costruito durante i nostri viaggi. Non è completa. Mostra solo le stanze di cui siamo a conoscenza. Se vuoi verificare che l'algoritmo funzioni, puoi seguire il processo di ricerca del percorso per costruire la tabella mostrata nella **Figura 6**.

CONSIGLIO RAPIDO

Se volessi costruire una mappa completa del castello e determinare il costo per raggiungere ogni stanza partendo dall'atrio, potresti semplicemente continuare a visitare le stanze finché non ne rimane più nessuna da visitare.

▲ **Figura 6:** Il percorso verso la Sala del Tesoro è diventato via via meno costoso man mano che esploravamo il castello.

▼ **Figura 7:** I numeri sui collegamenti non indicano le distanze; si tratta di costi associati alle porte.



Vincere con Dijkstra

L'ingegnosità dell'algoritmo emerge chiaramente quando si nota come abbia ignorato i luoghi che non rientravano nel percorso ottimale. Non abbiamo mai visitato le Stalle, poiché raggiungerle era più costoso rispetto alle alternative che avevamo preso in esame per prime. All'epoca in cui Dijkstra formulò il suo teorema, i computer disponevano di una memoria principale molto limitata. La sua dimostrazione di ricerca del percorso si limitava a 64 località, poiché il programma impiegava sei bit di dati per memorizzare l'identificativo di ognuna. Gran parte del successo iniziale di Dijkstra risiedette proprio nella capacità di far rientrare tutte le informazioni sulle località e il codice stesso in uno spazio di memoria estremamente ridotto.

Scrivere Dijkstra

Invece di usare un quaderno per tenere traccia dei costi, possiamo creare un nostro programma di monitoraggio dei costi in Python. La prima cosa da fare è creare un sistema per memorizzare ogni stanza:

```
class NodeRouteData:
    def __init__(self, name, cost, route_back):
        self.name = name
        self.visited = False
        self.cost = cost
        self.route_back = route_back
```

Il codice qui sopra crea una classe chiamata **NodeRouteData**. Essa memorizza tutte le informazioni relative a una posizione nel castello. Chiamiamo ogni posizione "nodo" perché potremmo voler utilizzare questo programma per navigare elementi diversi dalle stanze di un castello. La classe contiene campi che corrispondono direttamente a ciò che abbiamo annotato nel nostro taccuino. Quando creiamo un'istanza della classe, definiamo il nome, il costo e il percorso di ritorno da quel nodo. Ora chiediamo all'utente da dove parte e dove è diretto:

```
start_name = input("Nome partenza: ")
end_name = input("Nome arrivo: ")
```

Le istruzioni precedenti leggono i nomi dei punti di inizio e di fine e li memorizzano in variabili chiamate **start_name** e **end_name**. Successivamente, dobbiamo creare un nodo per rappresentare la posizione iniziale. Non è necessario crearlo per la posizione finale; utilizziamo il valore in **end_name** per rilevare quando abbiamo trovato un percorso per arrivarci.

```
start_node = NodeRouteData(start_name, 0, None)
```

Il comando precedente crea un **NodeRouteData** chiamata **start_node** col nome di **start_name**. Il primo argomento al costruttore di **NodeRouteData** è il nome del nodo. Il secondo argomento è il costo per raggiungerlo. Per il nodo di partenza, questo valore è 0. Il terzo argomento è il percorso di ritorno (ovvero il luogo in cui ci dirigiamo per tornare al punto di partenza). Dato che questo è il punto di partenza, non abbiamo un percorso di ritorno, quindi questo valore è impostato su **None**. Ora dobbiamo inserire il nodo iniziale nella tabella. Possiamo utilizzare un dizionario Python, e il nome del nodo fungerà da chiave per individuarlo all'interno del dizionario.

```
nodes = {}
nodes[start_name] = start_node
```

Le istruzioni precedenti creano un dizionario vuoto chiamato **nodes** e aggiungono la stanza iniziale. Aggiungo un elemento a un dizionario specificando una chiave (che verrà usata per trovare l'elemento), come mostrato sopra. Ora possiamo iniziare a controllare le porte in **nodes** e aggiornare i costi. Utilizziamo una variabile chiamata **current_node** per fare riferimento al nodo su cui stiamo lavorando. All'inizio della nostra esplorazione, il nodo corrente sarà il nodo iniziale:

```
current_node = start_node
```

L'affermazione sopra crea **current_node** e lo imposta come **start_node**. Ora avviamo il nostro ciclo:

```
while current_node.name != end_name:
```

Qui si fa partire un ciclo **while** che farà in modo che il programma esamini i nodi finché non ne raggiunge uno con lo stesso nome della nostra destinazione finale. Poiché l'algoritmo sceglie sempre i percorsi a costo minore, nel momento in cui i due nomi coincideranno, avremo individuato il percorso più economico. All'interno del ciclo, faremo esattamente ciò che abbiamo fatto esplorando fisicamente il castello: cercheremo le porte e useremo le informazioni su di esse per aggiornare la nostra tabella. Potrebbe esserci più di una porta, quindi dobbiamo creare un ciclo di "lettura delle porte":

```
while True:
    name = input("Nome della stanza (vuoto per uscire): ")
    if name == "":
        break
```

Le istruzioni sopra creano un ciclo che inizia chiedendo il nome di una stanza sulla porta. Se il nome è una stringa vuota, non ci sono altre porte in questa posizione, quindi il codice usa un **break** per uscire dal ciclo di lettura delle porte. Se l'utente ha inserito un nome, il programma richiede il costo del percorso.

```
cost_string = input("Costo :")
cost = int(cost_string)
```

Questo imposta la variabile **cost_string** con la stringa digitata dall'utente e poi converte questa stringa in un numero intero memorizzato nella variabile **cost**. Ora possiamo utilizzare questo nuovo valore di costo per aggiornare la nostra tabella. La prima cosa da fare è calcolare il costo per raggiungere la stanza indicata sulla porta partendo dalla posizione iniziale. Tale valore sarà dato dalla somma del costo per raggiungere la stanza corrente e del costo indicato sulla porta:

```
total_cost = cost + current_node.cost
```

Ora disponiamo del nome di un nodo e del costo totale per raggiungerlo. Se si tratta di un nodo che non abbiamo ancora incontrato, dobbiamo aggiungerlo all'insieme dei nodi. Procediamo.

```
if name not in nodes:
    nodes[name]=NodeRouteData(name,
    total_cost,current_node)
```

Questo codice controlla il dizionario **nodes** per verificare se contiene il nodo con il nome digitato. Se la risposta è negativa, aggiunge un nuovo nodo al dizionario. Reimpostiamo il percorso e i valori di costo per il nuovo nodo. Se il nodo è già presente nel dizionario, dobbiamo verificare se abbiamo trovato un percorso più economico per raggiungerlo. Possiamo farlo aggiungendo una parte **else** alla condizione che abbiamo appena scritto. Questo codice verrà eseguito se il dizionario contiene un nodo con il nome presente sulla porta.

```
else:
    room=nodes[name]
    if total_cost < room.cost:
        room.cost = total_cost
        room.route_back = current_node
```

Il codice qui ottiene un riferimento alla stanza dal dizionario e confronta il costo della porta appena scoperta con il costo memorizzato per raggiungere quella stanza. Se il nuovo percorso è più economico, aggiorniamo i dati della stanza. È l'equivalente informatico del cancellare le voci di costo e percorso precedente nel nostro taccuino e scriverne di nuove.

Una volta note tutte le informazioni sulle porte di una stanza, l'utente inserirà un nome di stanza vuoto e il ciclo relativo alle stanze terminerà. Ora dobbiamo registrare di aver visitato questa stanza, in modo da non tornarci più:

```
current_node.visited=True
```

Ora che abbiamo aggiornato la tabella di instradamento, dobbiamo decidere dove andare dopo. Dobbiamo individuare il percorso più economico nella tabella e dirigerci verso la destinazione a cui esso conduce:

```
cheapest_node = None

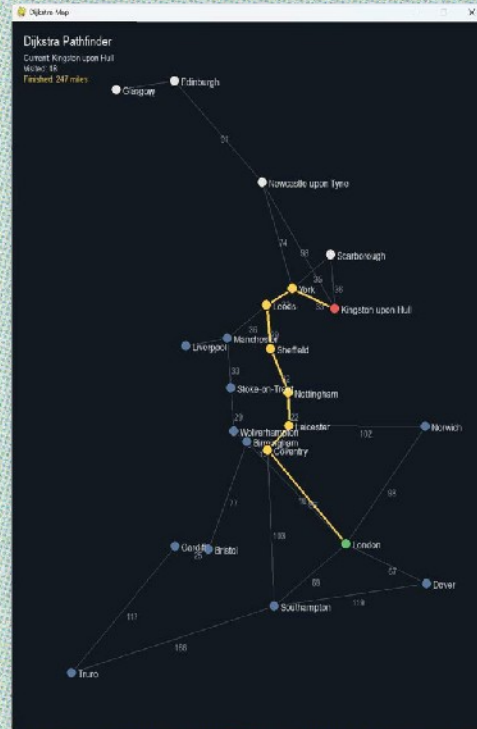
for node in nodes.values():
    if node.visited:
        continue
    if cheapest_node == None or node.cost <
    cheapest_node.cost:
        cheapest_node=node
```

Questo codice imposta il valore di **cheapest_node** al nodo in **nodes** che non è stato visitato e ha il costo più basso. Ora possiamo andarci:

```
print(f"Il nuovo nodo è {cheapest_node.name}")
current_node = cheapest_node
```

Se il nuovo percorso è
più conveniente,
aggiorniamo
i dati della stanza.

► **Figura 8:** Una mappa dei dati contenuti nel file `'map_graph.json'`. Il codice di esempio include un programma Python che disegna questa mappa e anima il processo di ricerca del percorso.



Il ciclo si ferma quando raggiunge l'inizio.

Queste due dichiarazioni indicano all'utente dove andare successivamente e quindi impostano il valore di `current_node` al più economico. Nota che il nodo successivo potrebbe non essere attraverso una delle porte che sono state appena scoperte. Ad un certo punto il ciclo si interromperà perché il `cheapest_node` diventerà quello con il nome finale e il ciclo `while` uscirà. È qui che comunichiamo all'utente il costo del percorso e glielo descriviamo. Visualizzare il costo del percorso è l'operazione più semplice delle due:

```
print(f"Costo: {current_node.cost}")
```

Il ciclo di ricerca si ferma quando `current_node` coincide con `end_name`. Quindi stampiamo semplicemente il costo per raggiungere il nodo corrente. Questo stamperà un costo di 18 monete. Ora costruiamo un percorso lavorando su tutti i valori `route_back` dei nodi sul percorso:

```
route = []

while current_node != None:
    route.append(current_node.name)
    current_node = current_node.route_back
```

Il codice qui sopra utilizza un ciclo per costruire una lista chiamata `route`. Inizia dal nodo finale e procede a ritroso fino all'inizio. Lo `start_node` è stato creato con un valore `route_back` pari a `None`, quindi il ciclo si fermerà quando raggiungerà l'inizio. Il ciclo aggiunge il nome di ogni nodo alla lista `route`.

Ora abbiamo un elenco di nomi di nodi, ma sono nell'ordine sbagliato. Dobbiamo invertirli prima di stamparli.

```
route.reverse()
print("Route:", " -> ".join(route))
```

Le due istruzioni qui invertono il percorso e poi stampano i nomi contenuti nel percorso, separati dalle stringhe `" -> "`. Il magnifico metodo `join` opera su una stringa, prende gli elementi di una collezione e inserisce la stringa tra ciascun elemento della collezione.

```
Hall -> Library -> Kitchen -> Dining Hall ->
Bedroom -> Treasure Room
```

L'output riportato sopra è il risultato di un'esecuzione riuscita. Il codice di esempio per questo articolo contiene una implementazione dell'algoritmo. Se sei arrivato fin qui, puoi fare un bel respiro e sentirti molto soddisfatto di te stesso. Ora sai come implementare l'algoritmo di Dijkstra. Inoltre, hai approfondito le tue conoscenze su classi, dizionari e riferimenti in Python.

map_graph.json

> Linguaggio: JSON

SCARICA IL
CODICE COMPLETO:rpmag.co/piconavGUARDA
A PAGINA

98

PER IL CODICE
COMPLETO**Navigazione su mappa**

Possiamo modificare il programma di navigazione "Castle" per tracciare percorsi nel Regno Unito. Ci serve un grafo che descriva i percorsi tra le località. Il termine "grafo" è usato in molti contesti diversi. Quello più familiare è probabilmente il grafico cartesiano che mette in relazione due valori (ad esempio, il prezzo in funzione del tempo). Per la pianificazione dei percorsi utilizziamo un grafo a nodi. Descrive luoghi (nodi) e i percorsi che li collegano. Ecco un esempio espresso sotto forma di file JSON.

Il codice nel file **map_graph.json** mostra la prima parte del grafo. Contiene un dizionario di città per nome. Le città sono caratterizzate da una posizione geografica e da un elenco di collegamenti e relativi costi (distanze) verso le città limitrofe. Una versione modificata del programma di navigazione "Castle" utilizza questo grafo a nodi per trovare il percorso più breve tra due città. Il Pico Route Finder impiega lo stesso codice per creare percorsi e visualizzarli poi sul display LCD.

CONSIGLIO RAPIDO

Se vogliamo lavorare su un problema diverso, ci basta modificare il grafo dei nodi.

```
001. {
002.   "cities": {
003.     "Birmingham": {
004.       "latitude": 52.4862,
005.       "longitude": -1.8904,
006.       "links": [
007.         {
008.           "destination": "Bristol",
009.           "cost": 77
010.         },
011.         {
012.           "destination": "Coventry",
013.           "cost": 17
014.         },

```

Dijkstra per sempre

L'algoritmo di Dijkstra funziona individuando in sequenza tutti i percorsi a costo minimo e arrestandosi una volta raggiunta la destinazione desiderata. Nel calcolare un itinerario da Londra a Hull, ad esempio, ha individuato un percorso per Truro (località molto distante sia da Londra che da Hull) prima di trovare quello per Hull. Per questo motivo, l'algoritmo di Dijkstra nella sua forma pura non viene utilizzato nei moderni sistemi di navigazione, che cercano di limitare la ricerca valutando se l'itinerario proposto avvicini o allontani dalla destinazione. Tuttavia, esistono molte situazioni in cui esso rappresenta ancora la soluzione migliore per definire un percorso e continuerà a essere parte integrante delle modalità con cui i sistemi informatici determinano gli spostamenti.

castle_nav.py

> Linguaggio: Python

SCARICA IL
CODICE COMPLETO:



rpimag.co/dijkstracode

```
001. class NodeRouteData:
002.     def __init__(self, name, cost, route_back):
003.         self.name = name
004.         self.visited = False
005.         self.cost = cost
006.         self.route_back = route_back
007.
008. start_name = input("Nome partenza: ")
009. end_name = input("Nome arrivo: ")
010.
011. start_node = NodeRouteData(start_name, 0, None)
012. nodes = {}
013. nodes[start_name] = start_node
014.
015. current_node = start_node
016.
017. while current_node.name != end_name:
018.     # Legge nel percorso
019.
020.     while True:
021.         name = input(
022.             "Nome stanza (vuoto per uscire): ")
023.         if name == "":
024.             break
025.
026.         cost_string = input("Costo :")
027.         cost = int(cost_string)
028.
029.         total_cost = cost + current_node.cost
030.         if name not in nodes:
031.             # nuova stanza - creiamola
032.             nodes[name] = NodeRouteData(
033.                 name, total_cost, current_node)
034.         else:
035.             # La stanza esiste
036.             # Questa porta offre un percorso
037.             # più conveniente?
```

```
035.         node = nodes[name]
036.         if total_cost < node.cost:
037.             # Vai! Percorso più economico
038.             node.cost = total_cost
039.             node.route_back = current_node
040.
041.         current_node.visited = True
042.
043.         # Trova la stanza più economica per
044.         # la prossima tappa.
045.         cheapest_node = None
046.
047.         for node in nodes.values():
048.             if node.visited:
049.                 continue
050.             if cheapest_node == None or
051.                 node.cost < cheapest_node.cost:
052.                 cheapest_node = node
053.
054.         print(f"Il prossimo nodo da visitare è
055.             {cheapest_node.name}")
056.
057.         current_node = cheapest_node
058.
059.         # Quando arriviamo qui, il percorso è già
060.         # stato scoperto.
061.
062.         print(f"Costo: {current_node.cost}")
063.
064.         route = []
065.
066.         while current_node != None:
067.             route.append(current_node.name)
068.             current_node = current_node.route_back
069.
070.         route.reverse()
071.         print("Percorso:", " -> ".join(route))
```